ByteMart Digital

THE INTERVIEW PREP SERIES

Microservices Interview Q&A

Architecture, communication, resilience, data, security, observability & deployment

112 interview questions with concise, accurate model answers, including 6 hands-on coding challenges with working solutions — organized by topic for focused preparation.

112	6	15	Model
Questions	Coding	Sections	Answers

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Microservices Interview Questions

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided in good faith for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from its use. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. All other product names and brands are the property of their respective owners and are used for identification purposes only. This is an independent work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

Fundamentals 4

Communication 7

Resilience Patterns 10

Data Management 13

Discovery, Config & Gateway 15

Security 17

Observability 19

Deployment & Scaling 21

Patterns & Anti-Patterns 23

Messaging & Testing 25

Design & Evolution 27

Reliability & Operations 29

Communication Deep-Dive 31

Scaling, Patterns & Practice 33

Coding & Config Challenges 37

This bank contains 112 Microservices interview questions across 15 sections, including 6 coding challenges with full solutions. Use the bookmarks panel to jump to any section or question.

SECTION 01

Fundamentals

Q1 What are microservices?

ANSWER

Microservices are an architectural style that structures an application as a collection of small, independently deployable services, each owning a specific business capability and communicating over lightweight protocols such as HTTP or messaging. Each service has its own codebase, its own database, and can be developed, deployed, and scaled independently by a small team.

The goal is not 'small' for its own sake but **independence** — teams can ship changes to one service without coordinating a release of the whole system, and services can scale and fail in isolation.

> **KEY IDEA**
> Microservices trade the simplicity of a single deployable for independent deployability, scalability, and team autonomy — at the cost of distributed-system complexity.

Q2 What is the difference between a monolith and microservices?

ANSWER

A **monolith** packages the entire application as one deployable unit with a shared database and in-process calls. **Microservices** split it into many services communicating over the network, each with its own data store.

	Monolith	Microservices
Deployment	one unit	many independent units
Scaling	whole app	per service
Data	shared database	database per service
Complexity	simpler ops	distributed complexity

> **TIP**
> A well-structured 'modular monolith' is often the right starting point; split out microservices when scaling, team, or deployment pressures justify the added complexity.

Q3 What are the main benefits and drawbacks of microservices?

ANSWER

- **Benefits** — independent deployment, targeted scaling, technology flexibility per service, fault isolation, and autonomous teams.

- **Drawbacks** — network latency and failures, distributed data consistency, operational overhead (monitoring, deployment), testing complexity, and harder debugging across services.

> **WATCH OUT**
> Microservices move complexity from the codebase into the infrastructure and the network — they are not free, and adopting them prematurely creates a distributed monolith that has the drawbacks of both.

Q4 What is a bounded context, and how does it relate to service boundaries?

ANSWER

A **bounded context** (from Domain-Driven Design) is a boundary within which a particular domain model and its terms have a single, consistent meaning. It is the natural guide for drawing service boundaries: each microservice should own one bounded context, so its model stays coherent and it changes for one business reason.

Aligning services with bounded contexts avoids the trap of splitting by technical layers, which creates chatty, tightly-coupled services that must all change together.

Q5 How do you decide the right granularity for a service?

ANSWER

Granularity is about cohesion and coupling, not lines of code. A good service owns a complete business capability, changes for one reason, and can be understood and operated by a single team. Too fine-grained, and you get network chattiness and distributed transactions; too coarse, and you lose independent deployability.

> **TIP**
> Favour fewer, well-bounded services early; split further only when a clear scaling, ownership, or deployment boundary emerges.

Q6 What is Conway's Law, and why does it matter for microservices?

ANSWER

Conway's Law observes that systems tend to mirror the communication structure of the organisation that builds them. For microservices this means service boundaries and team boundaries should align — each service owned by one team — otherwise cross-team coordination reintroduces the coupling microservices aim to remove.

© 2026 ByteMart Digital

Q7 What are the characteristics of a well-designed microservice?

ANSWER

- Owns a single business capability and its own data.
- Is independently deployable and versioned.
- Exposes a stable, well-documented API (a contract).
- Is loosely coupled and communicates asynchronously where possible.
- Is observable — emits logs, metrics, and traces.
- Is resilient — tolerates failures of its dependencies.

Q8 What is the difference between orchestration and choreography?

ANSWER

In **orchestration**, a central coordinator explicitly directs each service in a workflow — clear and easy to follow, but the orchestrator becomes a coupling point. In **choreography**, services react to events independently with no central controller — more decoupled and resilient, but harder to trace and reason about end to end.

> **TIP**
> Use orchestration for complex workflows needing clear control and compensation; use choreography for loosely-coupled, event-driven reactions.

SECTION 02

Communication

Q9 What is the difference between synchronous and asynchronous communication?

ANSWER

Synchronous communication (REST, gRPC) blocks the caller until the callee responds — simple and immediate, but it couples the services' availability: if the callee is slow or down, the caller suffers. **Asynchronous** communication (messaging, events) lets the caller continue without waiting, decoupling services in time and improving resilience, at the cost of eventual consistency and added complexity.

	Synchronous	Asynchronous
Coupling	temporal (both up)	decoupled in time
Consistency	immediate	eventual
Failure impact	propagates	buffered by broker

KEY IDEA

Prefer async messaging for cross-service workflows to avoid cascading failures; use sync calls for immediate request/response needs like a query.

Q10 When would you choose REST versus messaging?

ANSWER

Use **REST** (or gRPC) for request/response interactions where the caller needs an immediate answer — queries and simple commands. Use **messaging** (a broker like Kafka or RabbitMQ) for events, fire-and-forget work, and workflows spanning services, where decoupling and buffering matter more than immediacy.

Q11 What is service discovery, and why is it needed?

ANSWER

In a dynamic environment, service instances come and go and their network locations change. **Service discovery** is a registry (Eureka, Consul, or Kubernetes DNS) where services register themselves and look up others by logical name rather than hard-coded addresses — enabling elastic scaling and resilience.

In **client-side** discovery the caller queries the registry and load-balances itself; in **server-side** discovery a load balancer or the platform (Kubernetes Service) does it transparently.

© 2026 ByteMart Digital

Q12 What is an API Gateway?

ANSWER

An API Gateway is a single entry point that sits in front of the services and routes each request to the right one. It centralises cross-cutting concerns — authentication, rate limiting, TLS termination, request routing, and response aggregation — so individual services do not each reimplement them, and it hides the internal topology from clients.

> **WATCH OUT**
> Keep the gateway thin — pushing business logic into it recreates a monolith and a single point of failure. It should route and handle cross-cutting concerns, not domain rules.

Q13 What is client-side versus server-side load balancing?

ANSWER

Server-side load balancing puts a dedicated balancer (or the platform) between caller and instances, distributing requests centrally. **Client-side** load balancing has the caller pick an instance itself from the discovery registry (e.g. Spring Cloud LoadBalancer), removing an extra hop but pushing the logic into every client.

Q14 What is gRPC, and when would you use it over REST?

ANSWER

gRPC is a high-performance RPC framework using HTTP/2 and Protocol Buffers (a compact binary format) with a strongly-typed contract. It offers lower latency, smaller payloads, and streaming, making it attractive for high-throughput internal service-to-service calls, whereas REST/JSON remains the norm for public, browser-facing APIs.

Q15 What is the difference between Kafka and RabbitMQ?

ANSWER

Both are message brokers but with different models. **RabbitMQ** is a traditional message queue that routes and delivers messages to consumers and removes them once acknowledged — great for task queues and complex routing. **Kafka** is a distributed, durable log where messages are retained and consumers track their own position — great for event streaming, replay, and high throughput.

© 2026 ByteMart Digital

	RabbitMQ	Kafka
Model	queue / broker	distributed log
Retention	until consumed	configurable, retained
Best for	task queues, routing	event streaming, replay

Q16 What is event-driven architecture?

ANSWER

In event-driven architecture, services communicate by producing and consuming **events** — immutable facts about something that happened ('OrderPlaced'). Producers do not know who consumes their events, giving strong decoupling, natural scalability, and resilience. The trade-off is eventual consistency and the difficulty of reasoning about end-to-end flows.

SECTION 03

Resilience Patterns

Q17 What is the circuit breaker pattern?

ANSWER

A circuit breaker prevents a failing dependency from dragging down its callers. It monitors calls and, once failures cross a threshold, **opens** — short-circuiting further calls to fail fast (often returning a fallback) instead of piling up on a broken service. After a wait it goes **half-open** to test recovery, then **closes** if calls succeed.

State	Behaviour
Closed	calls pass through normally
Open	calls fail fast / fall back
Half-open	a few trial calls test recovery

KEY IDEA

The circuit breaker stops cascading failures by failing fast when a dependency is unhealthy, giving it time to recover.

Q18 What is the retry pattern, and what must you be careful about?

ANSWER

Retrying transparently re-attempts a failed call that might be transient (a brief network blip). It should use **exponential backoff with jitter** to avoid hammering a struggling service, cap the number of attempts, and only retry **idempotent** operations.

WATCH OUT

Retrying non-idempotent operations (like a payment) can cause duplicates, and synchronised retries across many clients can create a 'retry storm' that overwhelms a recovering service — jitter and circuit breakers mitigate this.

Q19 What is the bulkhead pattern?

ANSWER

Named after a ship's watertight compartments, the bulkhead pattern isolates resources (thread pools, connection pools) per dependency so that a failure or slowdown in one does not consume all resources and sink the whole service. If one downstream call hangs, only its bulkhead's threads are exhausted, leaving the rest of the service responsive.

© 2026 ByteMart Digital

Q20 What is a timeout, and why is it essential?

ANSWER

A timeout caps how long a call waits for a response before giving up. Without timeouts, a slow dependency ties up threads indefinitely, and the exhaustion cascades to callers — a leading cause of full-system outages. Every remote call should have a sensible, explicit timeout.

Q21 What is rate limiting, and where is it applied?

ANSWER

Rate limiting caps how many requests a client or the system will accept in a window, protecting services from overload and abuse. It is commonly enforced at the API gateway (per client/API key) using algorithms like token bucket or sliding window, returning HTTP 429 when the limit is exceeded.

Q22 What is a fallback, and how does it improve resilience?

ANSWER

A fallback is the alternative response returned when a call fails or a circuit is open — cached data, a default value, or a degraded feature. It lets the system fail **gracefully**, preserving core functionality (e.g. showing stale recommendations) rather than surfacing an error to the user.

Q23 What is Resilience4j?

ANSWER

Resilience4j is a lightweight Java fault-tolerance library providing composable modules — circuit breaker, retry, rate limiter, bulkhead, and time limiter — as annotations or decorators. It replaced the now-deprecated Hystrix and integrates cleanly with Spring Boot.

```java
@CircuitBreaker(name = "inventory", fallbackMethod = "fallback")
@Retry(name = "inventory")
public Stock check(Long id) { return inventoryClient.get(id); }

public Stock fallback(Long id, Throwable t) { return Stock.unknown(); }
```

Q24 What causes cascading failures, and how do you prevent them?

ANSWER

A cascading failure spreads when one overloaded or slow service causes its callers to block, exhaust threads, and fail in turn, propagating outward until the whole system is down. Prevention combines timeouts (never wait forever), circuit breakers (fail fast on unhealthy dependencies), bulkheads (isolate resources), and load shedding (reject excess work early).

© 2026 ByteMart Digital

SECTION 04

Data Management

Q25 What is the database-per-service pattern?

ANSWER

Each microservice owns its own private database, and no other service reads or writes it directly — access goes only through the owning service's API. This preserves loose coupling and lets each service choose the store that fits its needs, but it means data spanning services can no longer be joined in a single query or updated in one transaction.

> **WATCH OUT**
> A **shared database** across services is a well-known anti-pattern — it couples services at the schema level, so a change to one breaks others and independent deployment is lost.

Q26 Why are distributed transactions hard, and what is used instead?

ANSWER

A traditional ACID transaction across multiple service databases requires a two-phase commit, which is slow, blocking, and fragile at scale — and impossible when services use heterogeneous stores. Microservices instead embrace **eventual consistency**, coordinating multi-service operations with the **Saga** pattern rather than a single global transaction.

Q27 What is the Saga pattern?

ANSWER

A Saga breaks a business transaction that spans services into a sequence of **local** transactions, each in one service, coordinated by events or an orchestrator. If a step fails, the Saga runs **compensating** transactions to undo the completed steps, achieving consistency without a distributed lock.

It comes in two flavours: **choreography** (services react to each other's events) and **orchestration** (a central coordinator drives the steps and compensations).

```text
Order Saga (orchestrated):
  1. Create order (pending)
  2. Reserve payment  -> on failure: cancel order
  3. Reserve stock    -> on failure: refund payment, cancel order
  4. Confirm order
```

Q28 What is eventual consistency?

ANSWER

Eventual consistency means that after an update, the different copies of data across services will become consistent over time, but may be briefly out of sync. It is the pragmatic trade-off microservices accept in exchange for availability and decoupling, and it requires designing UX and logic to tolerate short windows of staleness.

Q29 What is CQRS?

ANSWER

Command Query Responsibility Segregation separates the **write** model (commands that change state) from the **read** model (queries), often with different data stores optimised for each. It lets reads scale independently and use denormalised views, at the cost of added complexity and eventual consistency between the two models.

Q30 What is event sourcing?

ANSWER

Instead of storing only the current state, event sourcing persists the full sequence of state-changing **events** as the source of truth; current state is derived by replaying them. This gives a complete audit trail and the ability to reconstruct past states, but adds complexity around event schema evolution and rebuilding projections.

Q31 What is the transactional outbox pattern?

ANSWER

To reliably publish an event when updating the database (avoiding the dual-write problem), the service writes the event to an **outbox** table in the same local transaction as the data change. A separate process then reads the outbox and publishes to the broker, guaranteeing the event is sent if and only if the data was committed.

Q32 How do services share data they don't own?

ANSWER

A service that needs another's data either queries it via API (synchronous, coupling availability) or keeps a **local read-only replica** updated by consuming the owner's events (asynchronous, decoupled). Event-driven replication is preferred for resilience, accepting eventual consistency of the copy.

© 2026 ByteMart Digital

SECTION 05

Discovery, Config & Gateway

Q33 How is configuration managed across microservices?

ANSWER

Configuration is externalised from the code (per the twelve-factor app) and centralised — for example with Spring Cloud Config backed by Git, or the platform's config maps and secrets. Services fetch environment-specific settings at startup (or refresh at runtime), so the same build runs everywhere and configuration changes do not require rebuilds.

Q34 How are secrets managed securely?

ANSWER

Secrets — credentials, API keys, certificates — must never live in code or plain config. They belong in a dedicated secrets manager (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets) that stores them encrypted, controls access, supports rotation, and audits usage. Services retrieve them at runtime.

Q35 What is a config server, and what does refresh scope do?

ANSWER

A config server centralises configuration and serves it to services on startup. With Spring Cloud, beans annotated `@RefreshScope` can pick up changed configuration at runtime (via an actuator refresh) without a restart — useful for tuning timeouts or toggling features live.

Q36 What are the responsibilities of an API Gateway?

ANSWER

- **Routing** — direct each request to the correct service.
- **Security** — authenticate and authorise before requests reach services.
- **Cross-cutting concerns** — rate limiting, TLS termination, CORS, logging.
- **Aggregation** — combine responses from several services for a client.
- **Resilience** — apply timeouts, retries, and circuit breaking at the edge.

© 2026 ByteMart Digital

Q37 What is a Backend for Frontend (BFF)?

ANSWER

A BFF is a gateway tailored to a specific client type — one for the web app, another for mobile — each shaping and aggregating data exactly as that client needs. It avoids a single gateway bloated with conditional logic for every client, at the cost of maintaining multiple edge services.

Q38 What is Spring Cloud Gateway?

ANSWER

Spring Cloud Gateway is a reactive, non-blocking API gateway built on Spring WebFlux. You define **routes** with predicates (path, header, method) and **filters** (rewrite paths, add headers, rate limit, circuit break). It integrates with service discovery and is the standard gateway in the Spring ecosystem.

yaml

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
```

SECTION 06

Security

Q39 How is authentication handled in a microservices architecture?

ANSWER

Authentication is centralised at the edge. A user authenticates once against an identity provider and receives a token (typically a JWT); the API gateway validates it, and the token is then propagated to downstream services, each of which trusts and inspects it. This avoids every service handling credentials directly.

> **KEY IDEA**
> Authenticate once at the edge, then propagate a signed token so each service can authorise independently without re-authenticating the user.

Q40 How does JWT propagation work between services?

ANSWER

The gateway forwards the caller's JWT in the `Authorization` header on internal calls, so each downstream service can validate the signature and read the claims (identity, roles) to make its own authorization decisions. The token carries the security context across the call chain without shared session state.

> **WATCH OUT**
> Downstream services must still validate the token, not blindly trust it — and internal traffic should be protected (e.g. mTLS) so tokens cannot be injected by a compromised service.

Q41 What are OAuth2 and OpenID Connect?

ANSWER

OAuth2 is an authorization framework that issues access tokens allowing a client to act on a resource on a user's behalf. **OpenID Connect (OIDC)** is an identity layer on top of OAuth2 that adds authentication and a standard ID token describing who the user is. Together they are the standard for delegated auth in modern systems.

Q42 What is mutual TLS (mTLS), and why use it internally?

ANSWER

In mutual TLS, both the client and the server present certificates and verify each other, so only trusted services can talk to one another. It secures service-to-service traffic against interception and impersonation and underpins zero-trust networking, often provided transparently by a service mesh.

© 2026 ByteMart Digital

Q43 What is the principle of zero trust?

ANSWER

Zero trust assumes the network is hostile and trusts nothing implicitly — every request, even internal, must be authenticated and authorised. It replaces the old 'hard perimeter, soft interior' model, so a breach of one service cannot freely move laterally through the system.

Q44 How do you secure service-to-service communication?

ANSWER

Combine several layers: encrypt traffic (TLS/mTLS), authenticate services (certificates or tokens), authorise each call (scopes/roles), and restrict network access (policies, mesh rules). Secrets are managed centrally, and every call is treated as untrusted until verified — defence in depth rather than a single perimeter.

© 2026 ByteMart Digital

SECTION 07

Observability

Q45 What are the three pillars of observability?

ANSWER

Observability rests on **logs** (discrete event records), **metrics** (numeric time-series like latency and error rate), and **traces** (the path of a single request across services). Together they let you not just detect that something is wrong but understand why in a distributed system where no single log tells the whole story.

Pillar	Answers
Logs	what happened, in detail
Metrics	how much / how often (trends)
Traces	where time went across services

Q46 What is distributed tracing, and how does it work?

ANSWER

Distributed tracing follows a single request as it flows through multiple services. A **trace id** is generated at the edge and propagated (via headers) to every service, each of which records a **span** for its portion of the work. Correlating spans reconstructs the end-to-end timeline, revealing which service caused latency or failure.

Tools like OpenTelemetry, Jaeger, and Zipkin collect and visualise these traces; Micrometer Tracing wires it into Spring Boot automatically.

Q47 What is a correlation id, and why is it important?

ANSWER

A correlation id is a unique identifier attached to a request at the entry point and carried through every service and log line it touches. Because logs are scattered across many services, the correlation id is what lets you gather all the logs for one user request into a single coherent story when debugging.

© 2026 ByteMart Digital

Q48 How is centralized logging implemented?

ANSWER

Each service writes structured (usually JSON) logs to stdout, and a collector (Fluentd, Logstash, or a platform agent) ships them to a central store (Elasticsearch/OpenSearch, Loki) where they are indexed and searchable through a dashboard (Kibana, Grafana). Centralisation is essential because logging into individual containers does not scale.

Q49 What metrics should a microservice expose?

ANSWER

Beyond system metrics (CPU, memory), expose the 'RED' signals — **Rate** (requests per second), **Errors** (failed requests), and **Duration** (latency distribution) — plus business metrics relevant to the domain. In Spring Boot, Micrometer exposes these to Prometheus via the Actuator.

Q50 What is the difference between health checks and metrics?

ANSWER

A **health check** is a simple up/down signal the platform uses to decide whether to route traffic or restart an instance (liveness and readiness). **Metrics** are continuous numeric measurements used for dashboards, alerting, and capacity planning. Health checks answer 'is it alive?'; metrics answer 'how is it performing?'.

Q51 What is a service mesh's role in observability?

ANSWER

A service mesh (Istio, Linkerd) runs a sidecar proxy alongside each service, transparently capturing traffic. Because all inter-service calls pass through it, the mesh can generate consistent metrics, traces, and access logs for every call **without changing application code** — plus it handles mTLS, retries, and routing.

© 2026 ByteMart Digital

SECTION 08

Deployment & Scaling

Q52 Why are containers a natural fit for microservices?

ANSWER

A container packages a service with its exact dependencies into a portable, isolated unit that runs identically in every environment. This solves the 'works on my machine' problem, makes services independently deployable and quick to start, and provides the standard unit that orchestrators schedule and scale.

Q53 What role does Kubernetes play?

ANSWER

Kubernetes is the container orchestrator that runs microservices at scale: it schedules containers across machines, restarts failed ones, scales replicas up and down, provides service discovery and load balancing, does rolling updates, and manages configuration and secrets. It automates the operational work that microservices would otherwise demand by hand.

Q54 What is a service mesh?

ANSWER

A service mesh is an infrastructure layer that handles service-to-service communication concerns — mTLS, retries, timeouts, load balancing, traffic shifting, and observability — via sidecar proxies, keeping this logic out of application code. It gives platform teams uniform control over networking policy across all services.

Q55 What are blue-green and canary deployments?

ANSWER

Blue-green runs two identical environments and switches all traffic from the old (blue) to the new (green) at once, enabling instant rollback. **Canary** releases the new version to a small percentage of traffic first, watching metrics before gradually increasing — limiting the blast radius of a bad release.

	Blue-green	Canary
Rollout	all at once (switch)	gradual %
Risk exposure	full after switch	limited subset
Rollback	instant switch back	stop and revert

© 2026 ByteMart Digital

Q56 What is the difference between horizontal and vertical scaling?

ANSWER

Vertical scaling gives an instance more resources (CPU, memory) — simple but bounded by machine size and requiring a restart. **Horizontal** scaling adds more instances and balances load across them — the microservices default, since stateless services scale out elastically and tolerate individual instance failures.

> **TIP**
> Design services to be **stateless** so they scale horizontally freely; push state to databases, caches, or the client.

Q57 What are the twelve-factor app principles (in brief)?

ANSWER

The twelve-factor methodology is a set of practices for building cloud-native services: externalise configuration, treat backing services as attached resources, keep processes stateless, export services via port binding, scale out via the process model, maximise dev/prod parity, and treat logs as event streams. Following them makes services portable, scalable, and disposable.

Q58 How does CI/CD support microservices?

ANSWER

With many independently deployable services, each needs its own automated pipeline that builds, tests, and deploys it on every change. CI/CD makes frequent, safe, independent releases possible — running the test suite, building a container, and rolling it out — which is precisely the independent deployability that justifies microservices.

© 2026 ByteMart Digital

SECTION 09

Patterns & Anti-Patterns

Q59 What is the strangler fig pattern?

ANSWER

The strangler fig pattern migrates a monolith to microservices incrementally: new functionality (and gradually extracted pieces of the old) is built as services, with a routing layer directing traffic to the new implementation as it is ready, until the monolith is 'strangled' and retired. It avoids a risky big-bang rewrite.

Q60 What is the sidecar pattern?

ANSWER

A sidecar is a helper container deployed alongside a service that provides supporting capabilities — logging, proxying, configuration, security — without the service implementing them. Service-mesh proxies are the classic example: they add networking and observability features transparently.

Q61 What is the aggregator pattern?

ANSWER

An aggregator (often in the gateway or a BFF) calls several services, combines their responses, and returns a single unified result to the client. It spares the client from making many calls and knowing the internal topology, at the cost of the aggregator becoming a coordination point.

Q62 What is a distributed monolith, and why is it bad?

ANSWER

A distributed monolith is the worst of both worlds: services that must be deployed together, share a database, or make tightly-coupled synchronous chains — so you pay the operational cost of distribution without gaining independent deployability. It usually results from splitting a system along the wrong (technical, not business) boundaries.

> **WATCH OUT**
> Symptoms include a change requiring coordinated releases of several services, shared database tables, and long synchronous call chains. The cure is realigning boundaries to bounded contexts and decoupling with events.

Q63 What is the ambassador pattern?

ANSWER

An ambassador is a proxy that handles outbound connectivity concerns for a service — retries, circuit breaking, routing, monitoring of calls to external systems — as a separate process, keeping that logic out of the application. It is a client-side counterpart to the sidecar.

Q64 Why is a shared database considered an anti-pattern?

ANSWER

When multiple services read and write the same database, they become coupled through its schema: a change one service needs can break the others, and none can be deployed or scaled truly independently. It reintroduces exactly the coupling microservices exist to remove, which is why database-per-service is the standard.

© 2026 ByteMart Digital

SECTION 10

Messaging & Testing

Q65 What is the difference between at-least-once and exactly-once delivery?

ANSWER

At-least-once delivery guarantees a message is delivered but may deliver it more than once (on retry), so consumers must be idempotent. **Exactly-once** is much harder and usually approximated by at-least-once delivery plus idempotent processing or deduplication. Designing consumers to be idempotent is the practical answer.

Q66 How do you make a message consumer idempotent?

ANSWER

Give each message a unique id and record processed ids (or use a natural business key with a uniqueness constraint), so reprocessing a duplicate has no additional effect. Idempotency is what makes at-least-once delivery safe and is essential for reliable event-driven systems.

```java
if (processedRepo.existsByMessageId(msg.id())) return;   // already handled
process(msg);
processedRepo.save(new Processed(msg.id()));
```

Q67 What is a dead letter queue?

ANSWER

A dead letter queue (DLQ) collects messages that repeatedly fail processing after the retry limit, so they do not block the main queue or get lost. They can then be inspected, fixed, and replayed. A DLQ is a safety net that keeps one poison message from stalling an entire consumer.

Q68 Why is message ordering challenging, and how is it addressed?

ANSWER

Across partitions and parallel consumers, messages can be processed out of order. Kafka guarantees order only **within a partition**, so you route related messages (e.g. all events for one order) to the same partition via a partition key. Where global ordering is impractical, design consumers to tolerate out-of-order events.

© 2026 ByteMart Digital

Q69 What is contract testing, and why does it matter?

ANSWER

Contract testing verifies that a service's API matches what its consumers expect, using a shared contract (e.g. with Pact or Spring Cloud Contract). It catches breaking API changes **before** deployment without the cost and fragility of full end-to-end integration tests, which is vital when services are released independently.

Q70 How do you test a microservice in isolation?

ANSWER

Test in layers: unit tests for logic; slice/component tests for the service with its dependencies stubbed (WireMock for HTTP, an embedded broker for messaging, Testcontainers for a real database); and contract tests for its API. Reserve a small number of true end-to-end tests for critical flows, since they are slow and brittle.

Q71 What is chaos engineering?

ANSWER

Chaos engineering deliberately injects failures — killing instances, adding latency, dropping network calls — into a system (often in production, carefully) to verify that resilience mechanisms actually work and to find weaknesses before real outages do. It turns resilience from an assumption into something continuously validated.

© 2026 ByteMart Digital

SECTION 11

Design & Evolution

Q72 What is the difference between SOA and microservices?

ANSWER

Both are service-oriented, but they differ in scale and coupling. Traditional **SOA** often relied on a heavyweight Enterprise Service Bus (ESB) with centralised orchestration and shared data. **Microservices** favour smart endpoints and dumb pipes — lightweight communication, decentralised data (database per service), and fine-grained, independently deployable services.

Q73 How do you version a microservice API without breaking clients?

ANSWER

Evolve additively — add optional fields and new endpoints rather than changing existing ones — and version only for genuinely breaking changes (URI or header versioning). Maintain the old version during a deprecation window so clients migrate on their own schedule, which is essential when services deploy independently.

> **WATCH OUT**
> Never make a breaking change to a live API that other services depend on without a versioning and migration plan — it causes cascading failures across teams.

Q74 What is an anti-corruption layer?

ANSWER

An anti-corruption layer is a translation boundary between your service's clean domain model and an external or legacy system's model. It converts data and calls at the edge so the foreign model's concepts do not leak in and corrupt your design — particularly valuable during a gradual migration from a legacy system.

Q75 What is a domain aggregate, and why does it matter?

ANSWER

In Domain-Driven Design, an aggregate is a cluster of related entities treated as a single consistency boundary, accessed through one root. Aggregates guide transaction boundaries within a service (one transaction per aggregate) and help identify natural service boundaries, keeping consistency local and avoiding distributed transactions.

© 2026 ByteMart Digital

Q76 How do you handle backward and forward compatibility in events?

ANSWER

Event schemas evolve, so consumers must tolerate change. Use a schema registry (e.g. with Avro/Protobuf), only add optional fields, never remove or repurpose existing ones, and have consumers ignore unknown fields. This lets producers and consumers deploy independently without a lockstep upgrade.

Q77 What is polyglot persistence?

ANSWER

Polyglot persistence means each service uses the database best suited to its needs — a relational store for transactional data, a document store for flexible documents, a search index for full-text, a time-series store for metrics. Database-per-service makes this possible, but it adds operational variety to manage.

© 2026 ByteMart Digital

SECTION 12

Reliability & Operations

Q78 What is the difference between an SLI, SLO, and SLA?

ANSWER

An **SLI** (Service Level Indicator) is a measured metric like availability or p99 latency. An **SLO** (Objective) is the internal target for that indicator (e.g. 99.9% availability). An **SLA** (Agreement) is the external, contractual commitment to customers, usually with penalties. SLOs drive engineering priorities; SLAs are the promises made to users.

Q79 What is load shedding?

ANSWER

Load shedding is deliberately rejecting or degrading low-priority requests when a service is overloaded, so it can keep serving critical traffic rather than collapsing entirely. Returning a fast 503 to excess requests protects the service's core function — a controlled failure beats an uncontrolled one.

Q80 What is backpressure?

ANSWER

Backpressure is a mechanism by which a consumer signals to a producer that it cannot keep up, so the producer slows down rather than overwhelming it. Reactive frameworks and bounded queues implement it, preventing unbounded buffering and out-of-memory failures under load.

Q81 What is graceful degradation?

ANSWER

Graceful degradation keeps a system partially functional when a dependency fails — for example, showing cached or default data, or hiding a non-essential feature, instead of returning an error. It preserves the core user experience under partial failure and is realised through fallbacks, caching, and feature toggles.

Q82 What is an idempotency key, and where is it used?

ANSWER

An idempotency key is a client-supplied unique identifier for an operation (often a header) that the server records, so retrying the same request returns the original result instead of performing the action twice. It makes non-idempotent operations like payments safe to retry over unreliable networks.

© 2026 ByteMart Digital

```text
POST /payments
Idempotency-Key: 5f3a1c9e-...
-> first call: charges and stores result under the key
-> retry with same key: returns the stored result, no second charge
```

Q83 What is change data capture (CDC)?

ANSWER

CDC streams a database's changes (inserts, updates, deletes) as events by reading its transaction log, letting other services or systems react to data changes without the owning service explicitly publishing them. Tools like Debezium implement it, and it is a robust way to feed read models and the outbox without dual writes.

Q84 What is the difference between liveness and readiness in Kubernetes?

ANSWER

A **liveness** probe checks whether the container is still functioning — failing it triggers a restart. A **readiness** probe checks whether it can accept traffic — failing it removes the pod from the load balancer without restarting. Getting these right avoids both routing traffic to a not-yet-ready pod and needlessly restarting a temporarily busy one.

© 2026 ByteMart Digital

SECTION 13

Communication Deep-Dive

Q85 What is the difference between choreography-based and orchestration-based Sagas?

ANSWER

In a **choreography** Saga, each service listens for events and reacts, publishing its own — fully decentralised but hard to trace. In an **orchestration** Saga, a central coordinator tells each service what to do and drives compensations — easier to understand and monitor, but the orchestrator is a coupling point. Choose based on workflow complexity and observability needs.

Q86 What are webhooks, and when are they used?

ANSWER

A webhook is a reverse API call: instead of a client polling for changes, the provider makes an HTTP callback to a registered URL when an event occurs. They are common for third-party integrations (payment confirmations, repository pushes), pushing events efficiently — but the receiver must verify authenticity and handle retries and duplicates.

Q87 What is the dual-write problem?

ANSWER

The dual-write problem arises when a service must update its database and publish an event, but the two are separate systems — a crash between them leaves them inconsistent (data saved but event lost, or vice versa). The transactional outbox pattern (or CDC) solves it by making the write and the event part of one atomic local transaction.

Q88 How do you propagate context (trace id, user) across async messages?

ANSWER

Carry context in message headers rather than the payload — the trace id, correlation id, and security context travel alongside the message so consumers can continue the trace and enforce authorisation. Instrumentation libraries (OpenTelemetry) automate propagation across both HTTP and messaging boundaries.

© 2026 ByteMart Digital

Q89 What is request collapsing / aggregation at the gateway?

ANSWER

Rather than a client making many calls, the gateway (or a BFF) fans out to several services, waits for their responses, and returns a single combined payload. This reduces client round-trips and latency over slow networks, at the cost of the gateway needing to know how to compose the data.

Q90 Why prefer asynchronous messaging for inter-service workflows?

ANSWER

Synchronous chains couple availability — if any service in the chain is down or slow, the whole request fails or stalls, and failures cascade. Asynchronous messaging decouples services in time: the broker buffers events, consumers process at their own pace, and a temporarily-down consumer simply catches up later, dramatically improving overall resilience.

> **KEY IDEA**
> Async messaging turns a chain of tightly-coupled synchronous calls into resilient, independently-paced event processing — the core of scalable microservice communication.

SECTION 14

Scaling, Patterns & Practice

Q91 When should you NOT use microservices?

ANSWER

Microservices add real cost — distributed complexity, operational overhead, and eventual consistency — so they are the wrong choice for small applications, early-stage products with unclear boundaries, or small teams. A well-structured modular monolith is usually the better starting point; extract services only when scaling, team size, or deployment friction genuinely demands it.

> **WATCH OUT**
> Adopting microservices prematurely, before the domain boundaries are understood, tends to produce a distributed monolith — the hardest architecture to change.

Q92 What are the common API rate-limiting algorithms?

ANSWER

The main algorithms trade precision against simplicity.

Algorithm	Behaviour
Token bucket	allows bursts up to a capacity, refills steadily
Leaky bucket	smooths output to a constant rate
Fixed window	count per fixed interval (edge bursts)
Sliding window	smoother, avoids window-edge bursts

Q93 What is the difference between stateless and stateful services?

ANSWER

A **stateless** service keeps no client state between requests — any instance can handle any request, so it scales horizontally and tolerates instance loss freely. A **stateful** service holds state (sessions, in-memory data) tied to specific instances, complicating scaling and failover. Microservices favour stateless designs, pushing state to databases or caches.

© 2026 ByteMart Digital

Q94 What is a distributed lock, and when do you need one?

ANSWER

A distributed lock coordinates access to a shared resource across service instances — for example ensuring only one instance runs a scheduled job. It is typically implemented with Redis (Redlock) or Zookeeper. Because distributed locks are hard to make correct under failures, prefer designs that avoid them (idempotency, partitioning) where possible.

Q95 What is the API composition pattern for queries?

ANSWER

When a query needs data owned by several services, an API composer calls each service and joins the results in memory. It is the query counterpart to the aggregator, avoiding a shared database — though it can be inefficient for large joins, in which case a CQRS read model that pre-joins the data is better.

Q96 What are multi-stage Docker builds, and why use them?

ANSWER

A multi-stage build uses one stage with the full build toolchain to compile the application and a second, minimal stage that copies only the resulting artifact into a small runtime image. This yields much smaller, more secure production images — no compilers or build dependencies shipped — which start faster and have a smaller attack surface.

Q97 What is event-carried state transfer?

ANSWER

In event-carried state transfer, events include the data a consumer needs, so the consumer can maintain its own local copy and avoid calling back to the producer. This maximises decoupling and availability (no synchronous dependency), at the cost of data duplication and eventual consistency of the replica.

Q98 How do you handle graceful shutdown in Kubernetes?

ANSWER

On termination, Kubernetes sends SIGTERM and removes the pod from the load balancer. The service should stop accepting new requests, finish in-flight ones within the grace period, and close resources cleanly. In Spring Boot, enabling graceful shutdown plus a readiness probe that flips to ‘not ready’ ensures traffic drains before the process exits.

© 2026 ByteMart Digital

Q99 What is the difference between orchestration platforms and a service mesh?

ANSWER

An orchestrator (Kubernetes) schedules, scales, and heals containers — it manages the lifecycle of services. A service mesh manages communication between them — mTLS, retries, traffic shifting, and observability — via sidecars. They are complementary layers: the orchestrator runs the pods, the mesh governs how they talk.

Q100 What is a feature flag, and how does it help deployment?

ANSWER

A feature flag wraps new functionality in a runtime toggle, so code can be deployed but kept dark until enabled — decoupling deployment from release. Flags enable canary rollouts to a subset of users, instant kill-switches for problematic features, and A/B testing, all without redeploying.

Q101 What is the difference between an aggregate query and a materialized view in CQRS?

ANSWER

An aggregate (API composition) query fetches and joins data from services on demand — always fresh but slower. A **materialized view** is a pre-computed read model, kept up to date by consuming events, that serves queries directly — fast reads at the price of eventual consistency and the effort of maintaining the projection.

Q102 How do you ensure only one instance runs a scheduled task?

ANSWER

Running the same `@Scheduled` job on every replica causes duplicate work. Coordinate with a distributed lock (e.g. ShedLock over the database or Redis) so only one instance executes each run, or delegate scheduling to an external scheduler/queue that dispatches the job to a single worker.

Q103 What is the difference between an API Gateway and a load balancer?

ANSWER

A **load balancer** distributes traffic across identical instances of one service at the network level. An **API Gateway** operates at the application level: it routes to different services by path, and adds cross-cutting concerns — auth, rate limiting, aggregation, transformation. A gateway often sits behind a load balancer and delegates instance-level balancing to it.

© 2026 ByteMart Digital

Q104 What is a composite (aggregate) health check?

ANSWER

A composite health check combines the health of a service's own critical dependencies — database, message broker, downstream APIs — into one overall status. Spring Boot Actuator aggregates individual `HealthIndicator`s this way, so the platform's readiness decision reflects whether the service can actually do its job, not just that the process is up.

Q105 What is the difference between throttling and rate limiting?

ANSWER

The terms overlap, but **rate limiting** typically means rejecting requests over a hard threshold (returning 429), while **throttling** often means slowing or queuing requests to smooth the load rather than rejecting them outright. Both protect a service from overload; rate limiting is stricter, throttling more forgiving.

Q106 What are consumer-driven contracts?

ANSWER

In consumer-driven contract testing, each consumer declares its expectations of a provider's API, and those contracts are verified against the provider in its build. This flips the usual direction — the provider learns exactly what consumers depend on and cannot break them unknowingly — enabling safe, independent evolution of services.

© 2026 ByteMart Digital

SECTION 15

Coding & Config Challenges

Q107 Configure a Resilience4j circuit breaker with a fallback.

</> CODE

SOLUTION

Annotate the call with `@CircuitBreaker` naming an instance and a fallback method with a matching signature plus a `Throwable`. Configure thresholds in properties.

java

```java
@Service
public class InventoryClientService {
    @CircuitBreaker(name = "inventory", fallbackMethod = "stockFallback")
    @Retry(name = "inventory")
    public StockDto getStock(Long productId) {
        return restClient.get()
            .uri("/stock/{id}", productId)
            .retrieve().body(StockDto.class);
    }

    public StockDto stockFallback(Long productId, Throwable t) {
        return new StockDto(productId, 0, "UNKNOWN");   // degrade gracefully
    }
}
```

yaml

```yaml
resilience4j:
  circuitbreaker:
    instances:
      inventory:
        sliding-window-size: 20
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
```

Q108 Declare a declarative HTTP client with Spring's HTTP interface.

</> CODE

SOLUTION

Spring 6's HTTP interface lets you declare a client as an annotated interface, and Spring generates the implementation — a modern replacement for OpenFeign in the Spring stack.

```java
public interface OrderApi {
    @GetExchange("/orders/{id}")
    OrderDto getOrder(@PathVariable Long id);

    @PostExchange("/orders")
    OrderDto create(@RequestBody CreateOrder body);
}

@Bean
OrderApi orderApi(RestClient.Builder builder) {
    RestClient client = builder.baseUrl("http://order-service").build();
    return HttpServiceProxyFactory
        .builderFor(RestClientAdapter.create(client))
        .build().createClient(OrderApi.class);
}
```

Q109 Configure retry with exponential backoff.

</> CODE

SOLUTION

Backoff spreads retries out and jitter avoids synchronised storms. Configure it declaratively in Resilience4j properties.

```yaml
resilience4j:
  retry:
    instances:
      inventory:
        max-attempts: 3
        wait-duration: 500ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
```

Q110 Define a Spring Cloud Gateway route with filters.

</> CODE

SOLUTION

Routes match requests by predicate and forward to a discovered service, applying filters for cross-cutting concerns such as stripping a prefix and rate limiting.

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
```

Q111 Publish and consume a domain event with Kafka in Spring.

</> CODE

SOLUTION

Produce with `KafkaTemplate` keyed for ordering, and consume with `@KafkaListener`, keeping the handler idempotent.

```java
// producer
kafkaTemplate.send("orders", order.id().toString(),
                   new OrderPlacedEvent(order.id(), order.total()));

// consumer
@KafkaListener(topics = "orders", groupId = "billing")
public void onOrderPlaced(OrderPlacedEvent event) {
    if (billingRepo.existsByOrderId(event.orderId())) return;  // idempotent
    billingService.invoice(event);
}
```

Q112 Sketch an orchestrated Saga with a compensating action.

</> CODE

SOLUTION

The orchestrator advances the workflow step by step, invoking a compensation if a later step fails so the overall operation stays consistent without a distributed transaction.

```java
public void placeOrder(OrderRequest req) {
    Long orderId = orderService.create(req);          // step 1
    try {
        paymentService.charge(orderId, req.amount());  // step 2
        inventoryService.reserve(orderId, req.items()); // step 3
        orderService.confirm(orderId);                  // step 4
    } catch (InventoryException e) {
        paymentService.refund(orderId);                 // compensate step 2
        orderService.cancel(orderId);                   // compensate step 1
        throw e;
    }
}
```